Appendix E

MACHINE LEARNING

Planned for publication in 2026 by R. Douglas Martin, Thomas K. Philips, Bernd Scherer and Kirk Li. All rights reserved. © Copyright 2025

10/22/25

Contents

E MACHINE LEARNING 1

E.1 Overview of Machine Learning 4

E.2 Supervised Learning Using Regularized Classical Algorithms 5

E.2.1 Regularization 6

E.2.2 Ridge Regression 8

E.2.3 Lasso 10

E.2.4 The Elastic Net 12

E.2.5 Cross-Validation 14

E.3 Robust Regularized Regression 15

E.3.1 Robust Ridge Regression 15

E.3.2 Robust Lasso and Elastic Net 16

E.4 Classification and Robust Logistic regression 17

E.4.1 Logistic Regression 18

E.5 Projection Pursuit Regression and Neural Networks 21

E.5.1 Projection Pursuit 21

E.5.2 Projection Pursuit Regression 26

E.1 Overview of Machine Learning

Machine Learning (ML), Large Language Models (LLMs), and Generative Artificial Intelligence (GAI) have had a significant impact on a variety of fields, and we fully expect portfolio management to prove no less vulnerable to their disruptive effects. Breiman (2001) identifies two disjoint statistical cultures:

1. A data–driven culture that is focused on modeling data as if it were drawn from a known data model, which is universally assumed to be a parametric distribution whose parameters can be estimated, ideally by maximum-likelihood, and fitting system responses (or dependent variables) to data items (or independent variables) using methods appropriate to the distribution (e.g. linear regression /least squares for jointly normal random variables), with goodness-of-fit being the primary figure of merit, and
2. A distribution–agnostic culture that is focused on identifying an (often high-dimensional) algorithmic relationship between the data and the system's responses, with forecast accuracy being the primary figure of merit.

Breiman (2001) argues that while most statisticians fall into the first group, it is the second group that is focused on what truly matters in the real world, and we might call members of the second group ML practitioners. Loosely speaking, ML refers to

1. Novel application of statistical and algorithmic methods and non-linear transformations to high-dimensional data,[1] as well as the use of non-linear models (e.g. multiplicative models, or models with interaction terms) in order to solve numerical problems such as forecasting future stock returns from current prices and financial statement information, and
2. Automatic model selection using regularized regression, as well as by applying measures of model complexity such as AIC (Akaike (1974)) and BIC (Schwarz (1978), as well as deLeeuw (1992)), and of predictive power, such as the Robust Final Predictor Error method covered in Chapter 6. This type of machine learning has been in use for over fifty years, and continues to be developed with new techniques for model selection even when the data sets are not huge, and when the number of variables used is modest.

LLMs and GAI, on the other hand, first transform, and then process, textual or visual data to solve unstructured problems such as summarizing news items, regulatory testimony, or management disclosures in corporate financial statements. ML, LLMs, and GAI should not be seen as mutually exclusive fields. They are better thought of as existing along a continuum, as the techniques and algorithms that underpin them have much in common.

Kelly and Xiu (2023) comprehensively survey the application of Machine Learning methods to problems in finance, and Hastie, R. Tibshirani, and J. H. Friedman (2009) and James et al. (2013) provide rigorous, insightful, and thoughtful coverage of the statistical aspects of machine learning. Gu et al. (2020) and Cao et al. (2024) exemplify the application of these methods to asset pricing and the estimation of expected returns, while Kim et al. (2023), Kim et al. (2024b) and Kim et al. (2024a) exemplify the application of Large Language Models and Generative Artificial Intelligence to the comprehension of financial statements.

The distinguishing features of machine learning methods are their ability to efficiently handle exceptionally large data sets with many explanatory factors, their focus on model selection and mitigating

[1] See for example, the material on identifying recesssions and recoveries in Chapter 11, where the raw unemployment rate must be transformed before it can be input to a threshold detector.

overfits to noisy data, and their ability to model the non-linearities that characterize most real-world problems.

A thoughtful investor must embrace both financial and technological perspectives when employing ML, LLMs, and GAI – the risk of lazy inquiry is the identification of models devoid of economic underpinnings that are modern day equivalents of constellations in a night sky studded with stars. The all-too-common consequence of such practices is models that generate good results when tested with available data, but which fail miserably when faced with new data. It is fair to say that the field of machine learning has grown so explosively in recent years that a portion of an appendix cannot do it justice, and we therefore focus on a small, but important, subset of such methods that are of particular relevance to portfolio management.

Machine Learning algorithms can usefully be classified into three categories:

1. Supervised learning: An algorithm is presented with a set of inputs or *features* (e.g. financial statements and market data) and outputs or *responses* (e.g. stock market returns) and is tasked with identifying the relationship between the two. Formally, given a collection of feature–response pairs $[\mathbf{X}, \mathbf{y}] = \{\mathbf{x}_i, \mathbf{y}_i\}$, $1 \leqslant i \leqslant N$, as well as a corresponding set of (usually non–negative) weights w_i and a loss function $\mathscr{L}$, find a high–dimensional function f such that $\sum_i w_i \mathscr{L}(f(\mathbf{x}_i) - \mathbf{y}_i)$ is minimized. It is often assumed, frequently without justification, that the features and the responses are linearly related to each other, and it good practice to explore the creation of new features by applying non-linear transformations to existing features.
2. Unsupervised learning: An algorithm is presented with a collection of objects (e.g., bonds), each with a set of characteristics (e.g., covenants, spreads, issuer's financial statements and history of defaults) and is tasked with estimating the high–dimensional joint density of the data, or of classifying it into a set of disjoint clusters such that the items in each cluster are close to each other under some metric, and are distant from items in other clusters. In the given example, the clusters might be ratings classes.
3. Reinforcement learning: An algorithm is presented with an initial set of inputs and an objective function which it is tasked with maximizing. New inputs are continually incorporated and consumed, and an optimal response is recomputed following each new observation. In this case, the reinforcement learning algorithm may be thought of as an optimal control policy, and can be applied to the multi-period optimization of a portfolio whose holdings have time-varying expected returns and transaction costs.

We restrict our exploration of machine learning to the field of *supervised learning* – a form of learning that is particularly relevant to portfolio management, and Least Squares and its variants prove to be a good starting point for many supervised learning algorithms, as does exploratory data analysis (EDA). The R packages `tourr` and `Pursuit` are particularly useful for EDA as they allow users to explore and visualize large multivariate datasets, project the data along different axes, and to tour the projections to identify clusters visually.

E.2 Supervised Learning Using Regularized Classical Algorithms

The classical linear model[2]

[2] We include the sizes of the various vectors and matrices as subscripts for clarity.

$$\mathbf{y} = \mathbf{X}\,\boldsymbol{\beta} + \boldsymbol{\epsilon} \tag{E.1}$$

which is used for time series and cross-section factor models in Chapters 6 and 7, and is described in detail in Appendix B minimizes $\boldsymbol{\epsilon}'\,\boldsymbol{\epsilon}$ and is given by

$$\begin{aligned}\hat{\boldsymbol{\beta}}_{LS} &= \underset{\boldsymbol{\beta}}{\operatorname{argmin}}\ \boldsymbol{\epsilon}'\,\boldsymbol{\epsilon} \\ &= \left(\mathbf{X}'_{p\times N}\,\mathbf{X}_{N\times p}\right)^{-1}\,\mathbf{X}'_{p\times N}\,\mathbf{y}_{N\times 1}.\end{aligned} \tag{E.2}$$

Equation (E.2) gives us insights into the limitations of Least Squares and its region of applicability.

1. If $p > N$, no unique solution exists because $\mathbf{X}'_{p\times N}\,\mathbf{X}_{N\times p}$ is singular. None the less, this is an important use case in genomics (see Johnstone and Titterington (2009) and Fan, Ma, et al. (2021)) and is increasingly important in finance, as researchers often throw every explanatory variable they have available into "kitchen sink" regressions in the hope of identifying a factor relevant to the problem at hand. This issue is addressed in depth in Chapter 11.
2. If the columns of $\mathbf{X}$ are nearly co-linear because some of the explanatory variables are closely related to, or constructed in almost the same way as others (e.g. Comprehensive Income, Net Income and Operating Income, all from the Income Statement of a firm's financial statements), $\mathbf{X}'_{p\times N}\,\mathbf{X}_{N\times p}$ becomes nearly singular, and the coefficient estimates become unstable.
3. Least squares does not perform variable selection: even if the true relationship between $\mathbf{X}$ and $\mathbf{y}$ is encapsulated in a small subset of the entries in $\boldsymbol{\beta}$, Least Squares will likely assign non-zero values to all its entries.
4. If the true relationship between $\mathbf{X}$ and $\mathbf{y}$ is non-linear, any transformation of the columns of $\mathbf{X}$ needs to be done ahead of time by the analyst. This, too, is addressed in Chapter 11, where it is shown that the addition of a quadratic term noticeably improves return forecasts for the S&P 500.
5. Least Squares is not robust to even a single outlier, and we refer the reader to Sections 2.7 and 6.2, where robust methods are introduced as data dependent weighted versions of least squares estimators, as well as to Chapter 16, whichdescribes robust regression algorithms that are largely unaffected by outliers, and which should be used routinely by financial analysts.

Some, though not all, of these problems can be addressed using *regularization* techniques which are designed to reduce the likelihood of overfitting models to data, and which we next discuss.

E.2.1 Regularization

The central idea that underlies regularization is simple: constrain the weights $\boldsymbol{\beta}$ in some way so that they become more stable and/or more sparse, and computable even if $p > N$. For notational simplicity, we assume that all the columns of $\mathbf{X}$ as well as $\mathbf{y}$ are demeaned, so that the first element of $\boldsymbol{\beta}$ corresponds to an explanatory variable, and not the intercept. This is standard practice when regularizing – it is variations from a baseline, not the baseline itself, that we are trying to fit, and shrinking the intercept will immediately introduce bias. Define

$$\begin{aligned} \|\epsilon\|^2 &= \epsilon'\epsilon. \\ &= (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})'(\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) \end{aligned} \tag{E.3}$$

and

$$\|\boldsymbol{\beta}\|_q = \sum_{i=1}^{p} |\boldsymbol{\beta}_i|^q, \quad q \geqslant 0. \tag{E.4}$$

While it is common practice to write$\|\mathbf{x}\|_q = \left(\sum_{i=1}^{p} |\mathbf{x}_i|^q\right)^{1/q}$, we have deliberately chosen not to raise the sum in (E.4) to the $1/q^{th}$ power as $q = 0$ is an important special case – $\|\boldsymbol{\beta}\|_0$ is just the number of non-zero components of $\boldsymbol{\beta}$.[3] When $q = 1$, it is the sum of their absolute values, and when $q = 2$, it is the square of the Euclidean length of $\boldsymbol{\beta}$. Regularization reformulates the least squares objective function to

$$\hat{\boldsymbol{\beta}}_q = \underset{\boldsymbol{\beta}}{\operatorname{argmin}} \ \|\epsilon\|^2 + \lambda_q \|\boldsymbol{\beta}\|_q, \quad q, \lambda_q \geqslant 0 \tag{E.5}$$

When $\lambda_q = 0$, we are back to classical Least Squares for all $q \geqslant 0$. When $q = 0$ and $\lambda_0 > 0$, we are trading off Mean Squared Error against the number of predictor variables. When $q = 1$ and $\lambda_1 > 0$, we are trading off Mean Squared Error against the sum of the magnitude of all the coefficients. This case is known as *lasso* (Least Absolute Shrinkage and Selection Operator, R. Tibshirani (1996), R. Tibshirani (2011)). When $q = 2$ and $\lambda_2 > 0$, we are trading off Mean Squared Error against the square of the Euclidean length of the coefficient vector. This case is known as *ridge regression* (Hoerl and Kennard (1970), Hastie (2020)).[4] The optimal value of λ is determined using *cross-validation*, which we discuss below.

Figure E.1 illustrates the shape of the constraint boundaries for lasso, ridge regression, and the elastic net, which is a weighted linear combination of the lasso and ridge penalties, with two explanatory variables. All three boundaries are convex, but only the latter two are strictly convex. The `R` package `glmnet` (`https://glmnet.stanford.edu/index.html`) provides efficient implementations of all these methods as well as many others.

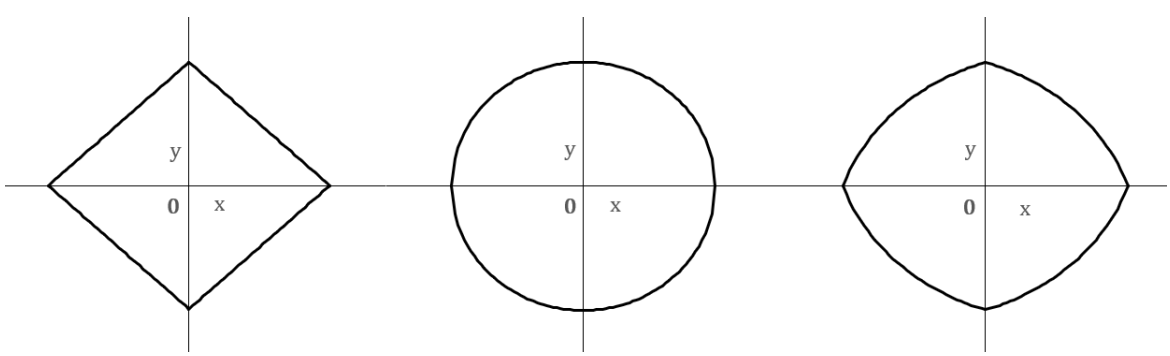

Fig. E.1 Shape of the Contraint Surfaces for Lasso, Ridge and Elastic Net For Two Variables

[3] It is standard practice in this context to assume that $0^0 = 0$.

[4] Technometrics devoted an entire issue (Vol 62, Issue 4, 2020) to celebrate the 50th anniversary of ridge regression.

E.2.2 Ridge Regression

The objective function for ridge regression can be written in two equivalent ways:

$$\hat{\boldsymbol{\beta}}_{ridge} = \underset{\boldsymbol{\beta}}{\operatorname{argmin}} \ \|\boldsymbol{\epsilon}\|^2 + \lambda \|\boldsymbol{\beta}\|_2, \ \ \lambda > 0 \tag{E.6a}$$

$$= \underset{\boldsymbol{\beta}}{\operatorname{argmin}} \ \|\boldsymbol{\epsilon}\|^2, \ \ \text{s.t.} \|\boldsymbol{\beta}\|_2 \leqslant c, \ c > 0 \tag{E.6b}$$

These formulations echo the two ways in which efficient frontiers can be computed (the Lagrangian formulation and the bound formulation), but the Lagrangian formulation (E.6a) is easier to deal with analytically. The bound formulation makes clear that $\boldsymbol{\beta}$ lies on or inside the surface of a hypersphere of radius $\sqrt{c}$. In fact, it mirrors the formulation of mean-variance efficient portfolio construction covered in Chapter 2.

Ridge regression was originally proposed by Hoerl and Kennard (1970) as an ad-hoc solution to the problem of collinearity in least squares. It is sometimes said to get its name from the fact that it adds a stabilizing "raised ridge" to the diagonal of $\mathbf{X}'\mathbf{X}$. The quadratic penalty term ensures that closed form solutions to (E.6a) and (E.6b) always exists, and it is left as an exercise to show that

$$\hat{\boldsymbol{\beta}}_{ridge} = \left(\mathbf{X}'_{p\times N}\mathbf{X}_{N\times p} + \lambda \mathbf{I}_{p\times p}\right)^{-1} \mathbf{X}'_{p\times N}\mathbf{y}_{N\times 1}, \tag{E.7}$$

that the p eigenvalues of $\mathbf{X}'_{p\times N}\mathbf{X}_{N\times p} + \lambda \mathbf{I}_{p\times p}$ can be obtained by adding λ to the eigenvalues of $\mathbf{X}'_{p\times N}\mathbf{X}_{N\times p}$, and that the eigenvectors of $\mathbf{X}'_{p\times N}\mathbf{X}_{N\times p} + \lambda \mathbf{I}_{p\times p}$ are identical to the eigenvectors of $\mathbf{X}'_{p\times N}\mathbf{X}_{N\times p}$. Simple rearrangement of (E.2) and (E.7) shows that

$$\hat{\boldsymbol{\beta}}_{ridge} = \left(\mathbf{I}_{p\times p} + \lambda \left(\mathbf{X}'_{p\times N}\mathbf{X}_{N\times p}\right)^{-1}\right)^{-1} \hat{\boldsymbol{\beta}}_{LS}. \tag{E.8}$$

In the special case when the columns of $\mathbf{X}$ are orthonormal (i.e. are orthogonal and and have unit variance), (E.8) reduces to

$$\hat{\boldsymbol{\beta}}_{ridge} = \frac{\hat{\boldsymbol{\beta}}_{LS}}{1+\lambda}. \tag{E.9}$$

Ridge regression is not linearly equivariant – the bracketed term in (E.7) makes clear that the regularization penalty, being additive, is proportionately larger for columns of $\mathbf{X}$ with low variance. It is the norm, therefore, to pre-process $\mathbf{X}$ ahead of solving (E.7) by standardizing all its columns so that they have a sample mean of 0 and a sample variance of 1. Also, ridge regression does not reduce the number of coefficients in a model – it simply shrinks them all towards zero and each other as λ increases. The optimal value of λ must be determined using cross-validation.

Hastie (2020) uses the Singular Value Decomposition (SVD) to show that if two or more of columns in $\mathbf{X}$ are identical, ridge regression will assign each one of them identical coefficients that sum to the same value that would be assigned to the single column that remains after all the duplicates are excised. Theobald (1974) proves that the covariance matrix of $\hat{\boldsymbol{\beta}}_{ridge}$ is smaller than that of $\hat{\boldsymbol{\beta}}_{LS}$ for some value of $\lambda > 0$. It follows that the ridge estimator is necessarily biased, as the Gauss-Markov Theorem (see Appendix B) assures us that least squares is the best linear unbiased estimator of $\mathbf{y}$ and minimizes the

covariance matrix of $\hat{\boldsymbol{\beta}}_{LS}$. When λ is small, ridge regression is almost identical to least squares, and its estimates are essentially unbiased. As λ increases, the coefficients are shrunk to 0: their variance declines, but their bias increases. At the optimal value of λ, the MSE, which is the sum of the variance and the square of the bias, is minimized.

There is a deep link between ridge regression and the Singular Value Decomposition.[5] If we write

$$\mathbf{X}_{N\times p} = \mathbf{U}_{N\times N}\mathbf{D}_{N\times p}\mathbf{V}'_{p\times p}, \tag{E.10}$$

with

$$\mathbf{U}'\mathbf{U} = \mathbf{V}'\mathbf{V} = \mathbf{I}, \tag{E.11}$$

and $\mathbf{D}_{N\times p}$ diagonal in the sense that $\mathbf{D}_{11} \geqslant \mathbf{D}_{22} \geqslant \mathbf{D}_{22} \geqslant \mathbf{D}_{\min(N,\, p),\, \min(N,\, p)} \geqslant 0$, and substitute E.10 into E.7 and simplify, we get

$$\hat{\boldsymbol{\beta}}_{ridge} = \sum_{i\ s.t.\ \mathbf{D}_{ii}>0} \mathbf{v}_i \frac{\mathbf{D}_{ii}}{\mathbf{D}_{ii}^2+\lambda}\mathbf{u}_i \cdot \mathbf{y} \tag{E.12}$$

where $\mathbf{u}_i$ and $\mathbf{v}_i$ are the left and right singular vectors of $\mathbf{X}$. The fitted values are easily seen to be

$$\begin{aligned}\hat{\mathbf{y}} &= \mathbf{X}\hat{\boldsymbol{\beta}}_{ridge}\\ &= \mathbf{U}\mathbf{D}\mathbf{V}'(\mathbf{D}'\mathbf{D} + \lambda\mathbf{I})^{-1}\mathbf{V}\mathbf{D}\mathbf{U}'\mathbf{y}\\ &= \sum_{i\ s.t.\ \mathbf{D}_{ii}>0} \mathbf{u}_i \frac{\mathbf{D}_{ii}^2}{\mathbf{D}_{ii}^2+\lambda}\mathbf{u}_i \cdot \mathbf{y}.\end{aligned} \tag{E.13}$$

Once the SVD of $\mathbf{X}$ has been identified and the inner products $\mathbf{u}_i \cdot \mathbf{y}$ computed, the entire ridge solution is known. The SVD representation of ridge regression makes clear that:

1. Small singular values are shrunk more than large singular values, and
2. The large singular values do not necessarily contribute to the prediction – the dot product of $\mathbf{u}_i$ and $\mathbf{y}$is just as important as the magnitude of $\mathbf{D}_{ii}$.

The second observation has an important practical implication: care must be taken when discarding terms in (E.12) and (E.13). The common practice of discarding small eigenvalues (in Principal Components Regression) or singular values (in SVD) can result in the inadvertent deletion of precisely those components of the data in which its explanatory power is most highly concentrated.

Projection Pursuit Analysis (PPA) and Independent Components Analyis (ICA) provide useful alternatives to PCA and SVD in such situations, as they identify directions along which the separation between classes is maximized.

As $\lambda \longrightarrow 0$, (E.12) and (E.13) converge to the OLS solution *even when the system of equations is under determined* (i.e. when $p > N$). In this case (E.12) and (E.13) represent the OLS solution that minimizes $\|\boldsymbol{\beta}\|_2$.

In spite of its simplicity and its inability to select a subset of the explanatory variables, ridge regression has proved to be a powerful way in which to balance bias and variance while minimizing MSE,

[5] See Appendix A for a discussion of the SVD.

and it plays a fundamental role in data science and machine learning. Hastie (2020) and Wieringen (2023) usefully collect five decades of insights into its many benefits, as well as its relationship to other concepts in Bayesian statistics and data science.

E.2.3 Lasso

Lasso [6] replaces ridge regression's quadratic penalty with a linear (in absolute value) penalty. As in the case of ridge regression, the objective function for lasso can be written using either a Lagrangian or a bound formulation:

$$\hat{\boldsymbol{\beta}}_{lasso} = \underset{\boldsymbol{\beta}}{\operatorname{argmin}} \ \|\epsilon\|^2 + \lambda \|\boldsymbol{\beta}\|_1, \ \ \lambda > 0 \tag{E.14a}$$

$$= \underset{\boldsymbol{\beta}}{\operatorname{argmin}} \ \|\epsilon\|^2, \ \ \text{s.t.} \|\boldsymbol{\beta}\|_1 < c, \ c > 0 \tag{E.14b}$$

In the special case when the columns of $\mathbf{X}$ are orthonormal, it can be shown that the solution to (E.14a) is

$$\hat{\boldsymbol{\beta}}_{lasso} = \left(\left| \hat{\boldsymbol{\beta}}_{LS} \right| - \frac{\lambda}{2} \mathbf{1} \right)^{+} \odot \operatorname{sgn}\left(\hat{\boldsymbol{\beta}}_{LS} \right), \tag{E.15}$$

where $\mathbf{1}$ is a column vector of p 1's, $x^+ = \max(x, 0)$, $\odot$ denotes the element-wise (or Hadamard) product of the two vectors, and $\operatorname{sgn}(x) = 1 \ \forall \ x \geqslant 0$ and -1 otherwise.

Turning to Chapter 5 for insight, we see that the computation of $\hat{\boldsymbol{\beta}}_{lasso}$ is a constrained quadratic program that can be solved using the computationally efficient LARS (Least Angle Regression) algorithm first described in Efron et al. (2004). R. J. Tibshirani (2013) proves that the solution to (E.14a) and (E.14b) is unique with probability 1 if the entries of $\mathbf{X}$ are drawn from a continuous distribution, and also describes an enhanced LARS algorithm that is implemented in the `R` package `glmnet`, which implements a wide range of linear models. Even when the entries of $\mathbf{X}$ are not drawn from a continuous distribution, and if two or more solutions exist, they are equivalent in that they have the same optimal value and level of sparsity.

The optimal value of λ must be chosen using cross-validation. The bound formulation (E.14b) makes clear that $\boldsymbol{\beta}$ lies on a surface defined by intersecting planes. The sharp corners at the intersections of these planes make the objective function not differentiable at these points and induce a sparsity property – unlike ridge regression, lasso naturally selects a subset of the explanatory variables and assigns zero weight to the others. Its great disadvantage (see Breiman (1996)) is that it can be unstable – both $\|\epsilon\|^2$ and $\hat{\boldsymbol{\beta}}_{lasso}$ can change dramatically when small changes are made to the selected subset of explanatory variables, or when new cases are added to the data.

Put differently, lasso lacks the *oracle property*: if the true model is based on a subset $\mathcal{P}$ of the available regressors with cardinality $p' < p$, then as $N \longrightarrow \infty$, an oracular algorithm will exhibit the following features:

[6] Lasso is an acronym for Least Absolute Shrinkage and Selection Operator.

$$\hat{\boldsymbol{\beta}}_i,\ i \notin \mathcal{P} = 0 \tag{E.16a}$$

$$\sqrt{N}\left(\hat{\boldsymbol{\beta}}_{\mathcal{P}} - \boldsymbol{\beta}_{\mathcal{P}}\right) \longrightarrow N\left(\mathbf{0},\ \boldsymbol{\Sigma}^*\right), \tag{E.16b}$$

where $\boldsymbol{\Sigma}^*$ is the covariance matrix of the true subset model.

The details are technical; we refer the reader to Fan and Li (2001) and Zou (2006) for a careful exploration of the oracle property as well as the relevant proofs. Zou (2006) shows that the *adaptive lasso* achieves the oracle property if the terms in the lasso's penalty function are suitably weighted, i.e.

$$\hat{\boldsymbol{\beta}}_{adaptive\ lasso} = \underset{\boldsymbol{\beta}}{\operatorname{argmin}}\ \|\epsilon\|^2 + \lambda \sum_{i=1}^{p} w_i \left|\boldsymbol{\beta}\right|_i,\ \ \lambda > 0 \tag{E.17}$$

where the weights w_i are derived from the coefficients of some other $\sqrt{N}-$consistent estimator of $\boldsymbol{\beta}$ such as ridge regression. Zou (2006) uses both LS and ridge weights, and defines

$$w_i = \left(\frac{1}{\left|\boldsymbol{\beta}_{\sqrt{N}-consistent,\ i}\right|}\right)^{\gamma},\ \gamma > 0. \tag{E.18}$$

In essence, when the coefficients of the ancillary estimator are small, the penalty in the lasso is amplified, so that the entries of $\hat{\boldsymbol{\beta}}_{adaptive\ lasso}$ corresponding to noise are more likely to be set to 0. Both γ and λ must be determined empirically using cross-validation.

Lasso is far more efficient computationally than the natural way in which to maximize sparsity, which is to identify the best subset of the explanatory variables. The best subset approach creates all $2^m - 1$ possible subsets of the m explanatory variables, regresses $\mathbf{y}$ against each one of these subsets, and finally selects the subset with smallest forecast error. The complexity of best subset selection grows rapidly with both m and N, though Bertsimas et al. (2016) describe an efficient mixed integer programming algorithm that solves the best subset problem for $m \approx 100$ and $N \approx 1000$ in a few minutes. The primary advantage of the best subset selection approach is that it cleanly eliminates noise variables that might display some spurious correlation with $\mathbf{y}$. Hastie, R. Tibshirani, and R. J. Tibshirani (2020) compare lasso to best subset as well as a variant of lasso known as *relaxed lasso* in which lasso is first used to identify the relevant variables, and least squares is then used to determine their coefficients.

Formally, let S_λ be the set of variables chosen by lasso for a given value of lambda, and let $\hat{\boldsymbol{\beta}}_{LS}\left(S_\lambda\right)$ be the least squares coefficient vector when $\mathbf{y}$ is regressed against the subset of columns of $\mathbf{X}$ corresponding to S_λ. To ensure that $\hat{\boldsymbol{\beta}}_{LS}\left(S_\lambda\right)$ has length p, we pad it with zeroes in positions corresponding to variables that are not selected. The relaxed lasso estimator is a weighted average of the pure lasso estimate and the restricted least squares estimate, i.e.

$$\hat{\boldsymbol{\beta}}_{relaxed\ lasso}\left(\lambda,\ \gamma\right) = \gamma\,\hat{\boldsymbol{\beta}}_{lasso} + (1-\gamma)\hat{\boldsymbol{\beta}}_{LS}\left(S_\lambda\right),\ 0 < \lambda,\ 0 \leqslant \gamma \leqslant 1. \tag{E.19}$$

The optimal values of λ and γ must be determined from cross-validation, and an upper bound on λ can be obtained by testing a few cases till one is found that forces all (or almost all) the coefficients to 0.

Lasso, unlike ridge regression, tends to perform poorly when two or more of the variables are highly correlated. The total weight assigned to the highly correlated group can be distributed in very different proportions among the correlated variates with little impact on the objective function. Ridge regression, on the other hand, will tend to weight them equally as its quadratic penalty diminishes when there is an even allocation between highly correlated variables.

This observation is of practical importance: it sometimes proves necessary to include or exclude a complete (predefined) set of columns of $\mathbf{X}$, as, for example, when a categorical variable is encoded by a set of l dummy variables, the ith one of which is 1 (all the other $l-1$ are 0) according to whether or the particular observation is associated with category i. The grouping problem arises naturally in machine learning, and Yuan and Lin (2006) solve it using a variant of lasso they call group lasso.

Hastie, R. Tibshirani, and R. J. Tibshirani (2020) conduct a series of experiments comparing lasso and relaxed lasso to best subset selection and forward stepwise selection.When the signal-to-noise ratio (SNR) is low, lasso tends to perform better than best subset selection and forward stepwise selection. When it is high, the reverse is true. Relaxed lasso turns out to be the overall winner – it performs about as well as lasso when SNR is low, and about as well as best subset selection when it is high. While no single method uniformly dominates the others, it is important to keep in mind that it is the low SNR case that is of greatest relevance to finance, making lasso and relaxed lasso useful starting points for machine learning in finance.

E.2.4 The Elastic Net

Zou and Hastie (2005) approach the correlated variable problem by combining the ridge and lasso penalties to get a new penalty that they call the *elastic net*. While it is not a complete cure for the problems that afflict both ridge regression and lasso, the elastic net inherits both the sparsity of lasso and the ability of ridge regression to preserve and equally weight groups of correlated variables. Zou and Hastie (2005) colorfully describe the elastic net as a stretchable fishing net that "catches all the big fish" while letting the little ones slip away, and start by formulating the *naïve elastic net*, which solves

$$\begin{aligned}\hat{\boldsymbol{\beta}}_{\textit{naïve elastic net}} &= \underset{\boldsymbol{\beta}}{\operatorname{argmin}} \|\epsilon\|^2 + \lambda_1 \|\boldsymbol{\beta}\|_1 + \lambda_2 \|\boldsymbol{\beta}\|_2, \;\; \lambda_1, \lambda_2 > 0 \\ &= \underset{\boldsymbol{\beta}}{\operatorname{argmin}} \|\epsilon\|^2 + \lambda \left(\alpha \|\boldsymbol{\beta}\|_1 + (1-\alpha) \|\boldsymbol{\beta}\|_2\right), \;\; \lambda > 0, \; 0 < \alpha < 1 \end{aligned} \tag{E.20}$$

The objective function for the naïve elastic net can be rewritten using both the Lagrangian and bound formulations:

$$\hat{\boldsymbol{\beta}}_{\textit{naïve elastic net}} = \underset{\boldsymbol{\beta}}{\operatorname{argmin}} \; \|\epsilon\|^2 + \lambda \left(\alpha \|\boldsymbol{\beta}\|_1 + (1-\alpha) \|\boldsymbol{\beta}\|_2\right), \;\; \lambda > 0, \; 0 < \alpha < 1 \tag{E.21a}$$

$$= \underset{\boldsymbol{\beta}}{\operatorname{argmin}} \; \|\epsilon\|^2, \;\; \text{s.t.} \left(\alpha \|\boldsymbol{\beta}\|_1 + (1-\alpha) \|\boldsymbol{\beta}\|_2\right) < c, \; c > 0, \; 0 < \alpha < 1. \tag{E.21b}$$

and Zou and Hastie (2005) show that the solution to the naïve elastic net is identical to the solution of lasso on augmented data sets $\mathbf{X}^*$ and $\mathbf{y}^*$, where

$$\mathbf{X}^*_{(N+p)\times p} = \frac{1}{\sqrt{1+\lambda_2}} \begin{bmatrix} \mathbf{X}_{N\times p} \\ \sqrt{\lambda_2}\mathbf{I}_{p\times p} \end{bmatrix} \tag{E.22}$$

and

$$\mathbf{y}^*_{(N+p)\times 1} = \begin{bmatrix} \mathbf{y}_{N\times 1} \\ \mathbf{0}_{p\times 1} \end{bmatrix}. \tag{E.23}$$

Furthermore, if we let $\gamma = {}^{\lambda_1}/_{\sqrt{1+\lambda_2}}$, $\boldsymbol{\beta}^* = \sqrt{1+\lambda_2}\,\boldsymbol{\beta}$ and

$$\epsilon^*_{(N+m)\times 1} = \mathbf{y}^*_{(N+p)\times 1} - \mathbf{X}^*_{(N+p)\times p}\,\boldsymbol{\beta}^*_{(N+p)\times 1}. \tag{E.24}$$

Then

$$\hat{\boldsymbol{\beta}}_{\textit{naïve elastic net}} = \frac{1}{\sqrt{1+\lambda_2}} \times \left[\underset{\boldsymbol{\beta}^*}{\operatorname{argmin}} \|\epsilon^*\|^2 + \gamma \left\|\boldsymbol{\beta}^*\right\|^1\right], \ \gamma > 0, \tag{E.25}$$

so that the naïve elastic net is just lasso applied to an augmented data set!

Like lasso, the elastic net has sharp boundaries, and does both variable selection and shrinkage, and in the special case when the columns of $\mathbf{X}$ are orthonormal it can be shown that the solution to (E.25) is

$$\hat{\boldsymbol{\beta}}_{\textit{naïve elastic net}} = \frac{\left(\left|\hat{\boldsymbol{\beta}}_{LS}\right| - \frac{\lambda_1}{2}\mathbf{1}\right)^+ \odot \operatorname{sgn}\left(\hat{\boldsymbol{\beta}}_{LS}\right)}{1+\lambda_2}. \tag{E.26}$$

On comparing (E.9), (E.15) and (E.26) is clear that ridge regression shrinks but does not select, lasso selects and thresholds but does not shrink, and the naïve elastic net does both. Less obviously, the naïve elastic net does both, and in addition, has a grouping property – like ridge regression, it gives highly correlated variables similar weights. If the columns of $\mathbf{X}$ are standardized to have mean 0 and variance 1, and if the ith and j^{th} components of $\hat{\boldsymbol{\beta}}_{\textit{naïve elastic net}}$ are positively correlated, the magnitude of the difference between them is bounded by

$$\left|\hat{\boldsymbol{\beta}}_{\textit{naïve elastic net},\, i} - \hat{\boldsymbol{\beta}}_{\textit{naïve elastic net},\, j}\right| \leqslant \frac{\|\mathbf{y}\|_1}{\lambda_2}\sqrt{2\,(1-\rho)}, \tag{E.27}$$

where ρ is the sample correlation between the ith and j^{th}columns of $\mathbf{X}$ (see Theorem 1 and Appendix A of Zou and Hastie (2005) for a proof).

In practice, though, the naïve elastic net tends to shrink too much – its solution is first impacted by λ_2 (which induces shrinkage as does ridge regression when the columns of $\mathbf{X}$ are orthogonal – see (E.9)) and then by λ_1, which induces further lasso-like selection. Zou and Hastie (2005) suggest a pragmatic solution to this problem – they undo the ridge-like shrinkage by scaling the coefficients of the naïve elastic net by $1+\lambda_2$ and transform the naïve elastic net into the *elastic net*:

$$\begin{aligned}\hat{\boldsymbol{\beta}}_{\textit{elastic net}} &= (1+\lambda_2) \times \hat{\boldsymbol{\beta}}_{\textit{naïve elastic net}} \\ &= \sqrt{1+\lambda_2} \times \left[\underset{\boldsymbol{\beta}^*}{\operatorname{argmin}} \|\epsilon^*\|^2 + \gamma \left\|\boldsymbol{\beta}^*\right\|_1\right], \ \gamma > 0.\end{aligned} \tag{E.28}$$

By doing so, they preserve the variable selection property of lasso and induce the desirable variable grouping property we earlier described. Zou and Zhang (2009) show that a variant of (E.28) that they call the *adaptive elastic net* exhibits the oracle property. As in the case of the adaptive lasso, they first identify $\hat{\boldsymbol{\beta}}_{\textit{elastic net}}$, and then use these coefficients to weight the individual components of $\left\|\boldsymbol{\beta}^*\right\|_1$ in a second model

$$\hat{\boldsymbol{\beta}}_{adaptive\ elastic\ net} = (1+\lambda_2) \times \underset{\boldsymbol{\beta}}{\operatorname{argmin}} \|\boldsymbol{\epsilon}\|^2 + \lambda_1 \sum_{i=1}^{p} w_i \left|\boldsymbol{\beta}\right|_i + \lambda_2 \|\boldsymbol{\beta}\|_2, \ \ \lambda_1, \lambda_2 > 0 \tag{E.29}$$

where

$$w_i = \left(\frac{1}{\left|\boldsymbol{\beta}_{elastic\ net,\ i}\right|} \right)^{\gamma}, \ \gamma > 0. \tag{E.30}$$

As in the case of the adaptive lasso, small elastic net coefficients amplify the corresponding terms penalty in the adaptive elastic net, making the removal of noise coefficients more likely. Both γ and λ must be determined empirically using cross-validation.

E.2.5 Cross-Validation

Cross-validation involves splitting the training data set into two or more parts, some of which are used for model building, and others that are used for model testing ahead of implementation.

Leave-out-one cross-validation is the simplest form of cross-validation, and is applicable to data without a temporal component (i.e. without a well defined notion of past, present and future). From a data set with N items, we create $N-1$ data sets with $N-1$ items each by removing the ith row of $\mathbf{X}$ and $\mathbf{y}$ for $1 \leqslant i \leqslant N$. Next, we build $N-1$ models using our preferred modeling technique, and use these to forecast the excluded observations $\mathbf{y}_i$ one at a time. The mean squared error is a function of both the modeling technique and λ and is given by

$$MSE\left(\lambda\right) = \frac{1}{N} \sum_{i=1}^{N} \left(\hat{\mathbf{y}}_i - \mathbf{y}_i \mid \lambda\right)^2 \tag{E.31}$$

where $\hat{\mathbf{y}}_i$ is the forecast of $\mathbf{y}_i$ made using the model built on the remaining $N-1$ rows of $\mathbf{X}$ and $\mathbf{y}$. The value of λ must be determined by minimizing MSE over all possible values of λ:

$$\lambda^* = \underset{\lambda}{\operatorname{argmin}}\ MSE\left(\lambda\right). \tag{E.32}$$

An upper bound on λ can be obtained by testing a few cases till one is found that forces all (or almost all) the coefficients to 0, and the cross-validation procedure can then examine a grid of values between 0 and λ_{max}. When N is so large that leave-out-one cross-validation is computationally prohibitive, cross-validation can also be done with a $k-$fold split: the data is broken up into $k << N$ subgroups of approximately equal size; $k-1$ of which are used as a training set, with the last one being used as a validation set. Breiman and Spector (1992) suggest that k be set to 5, though 10 is also commonly used.

When the data has a temporal component, a *training-validation-testing* data split is common: the data is split into three parts. The oldest part (the training set) is used to build models with a wide range of values for λ. The middle part (the validation set), is used to test each model built using the training set. Finally, the model (or the parameter λ) that gives rise to the best forecasts in the validation set is identified, and is then applied to the most recent part (the testing set). It is important that the testing data set be held out and completely unavailable to the algorithm during training and validation, as viewing it ahead of time is equivalent to looking into the future.

Parameter identification for temporal data can be done in other ways as well. For example, we use expanding window robust regressions in Chapter 11 to forecast the forward returns of the S&P 500 Index. Assume we have data for time periods $1,\ 2,\ \cdots,\ T$. We start by fitting a model using robust regression over some initial time period $1,\ 2,\ \cdots,\ T_0$, and then test its prediction for time period T_0+1. We then build a new model using data over an expanded window $1,\ 2,\ \cdots,\ T_0+1$ and test its prediction for time period T_0+2. Each period the training window expands by one to allow the incorporation of new data, and this process is continued till the data is exhausted. The entire loop can be repeated using different values of λ before choosing a final value of λ for use with the next incoming observation.

E.3 Robust Regularized Regression

Even though robust statistical methods have made inroads into statistical modeling and have seen some application in portfolio management as in Martin, Stoyanov, et al. (2023) and Martin, Guerard, et al. (2024), they are far from being a standard method in the field of machine learning. A notable exception is the use of a well-known Huber method of controlling for outliers in Gu et al. (2020). Even so, some robust analogues of the methods described in Section E.2 exist, and we expect many more to be developed in the near future. The R package `pense` has robust extensions of ridge, lasso and elastic net estimators, and a good discussion of these methods can be found in Chapter 5 of Maronna et al. (2019) as well as in Cohen Freue et al. (2019). We use it for all our work.

The central idea in robust regularization is to replace $\|\epsilon\|^2$ in the various objective functions with $\sum_i \rho\left(\epsilon_i/\hat{s}\right)$ for some symmetric loss function $\rho(.)$ and some robust estimator of scale $\hat{s}$.

E.3.1 Robust Ridge Regression

Maronna (2011) describes a robust algorithm for ridge regression that is best thought of as a weighted version of (E.6a) with data dependent weights $\mathbf{w}$, where $\mathbf{w}_i,\ 1 \leqslant i \leqslant N$ is the weight applied to the ith row of $\mathbf{X}$. If we let $\mathbf{W}_{N\times N} = diag(\mathbf{w})$, the robust equivalent of (E.6a) is

$$\hat{\mu},\ \hat{\boldsymbol{\beta}}_{robust\ ridge} = \left(\mathbf{X}'_{p\times N}\,\mathbf{W}_{N\times N}\,\mathbf{X}_{N\times p} + \lambda\,\mathbf{I}_{p\times p}\right)^{-1}\mathbf{X}'_{p\times N}\,\mathbf{W}_{N\times N}\,\left(\mathbf{y}_{N\times 1} - \hat{\mu}\mathbf{1}_{N\times 1}\right), \tag{E.33}$$

where $\hat{\mu},\ \hat{\boldsymbol{\beta}}_{robust\ ridge}$, and $\mathbf{W}_{N\times N}$ must be identified iteratively.

Earlier, we could safely assume that $\mathbf{y}$ was demeaned so that $\hat{\mu}$ could be set to 0, but in the presence of outliers, this is no longer true, and the algorithm must provide a robust estimate of its location. The weights

$$\mathbf{w}_i = \frac{\psi(\epsilon_i)}{\epsilon_i} \tag{E.34}$$

are determined by applying (18.49) to the residuals ϵ obtained from

$$\epsilon = \mathbf{y} - \hat{\mu}\,\mathbf{1}_N - \mathbf{X}\,\hat{\boldsymbol{\beta}}_{robust\ ridge}. \tag{E.35}$$

Maronna (2011) starts by using a highly robust (but potentially inefficient) regression estimator to obtain an initial estimate of the regression coefficients $\hat{\mu}_0$ and $\hat{\boldsymbol{\beta}}_0$, following which the residuals $\boldsymbol{\epsilon}_0$ are computed. He then obtains an initial estimate of their scale, $\hat{\sigma}_0$, by solving

$$\frac{1}{N}\sum_{i=1}^{N}\rho_0\left(\frac{\epsilon_{0,\,i}}{\hat{\sigma}_0}\right)=\delta, \tag{E.36}$$

where

$$\rho_0(x)=\rho_{Bisquare}\left(\frac{x}{c_0}\right)=\min\left(1,\ 1-\left(1-\left(\frac{x}{c_0}\right)^2\right)^3\right), \tag{E.37}$$

and the values of δ and c_0 determine the algorithm's robustness.

Next, using the initial estimate of scale, as well as an estimate of the equivalent degrees of freedom in the data, the algorithm adjusts c_0 upwards to a new value c that assures a desired level of normal efficiency when minimizing the loss function

$$\mathscr{L}\left(\mathbf{X},\ \mathbf{y},\ \mu,\ \boldsymbol{\beta}\right)=\hat{\sigma}_0^2\cdot\sum_{i=1}^{N}\rho\left(\frac{\mu,\ \epsilon_i\left(\boldsymbol{\beta}\right)}{\hat{\sigma}_0}\right)+\lambda\left\|\boldsymbol{\beta}\right\|_2, \tag{E.38}$$

where

$$\rho(x)=\rho_{Bisquare}\left(\frac{x}{c}\right)=\min\left(1,\ 1-\left(1-\left(\frac{x}{c}\right)^2\right)^3\right). \tag{E.39}$$

Finally, using weights derived from (E.34), (E.38) and (E.39), the iterative procedure repeatedly updates $\mathbf{W}$, $\hat{\boldsymbol{\beta}}_{robust\ ridge}$, and $\hat{\mu}$ till the changes in $\boldsymbol{\epsilon}$ are sufficiently small. The algorithm has a number of subtleties, starting with the fact that its robustness depends on both the initial estimate of scale and the effective number of parameters, and these are covered in Section 5.10 of Maronna et al. (2019).

We recommend implementing robust ridge regression using the function `regmest` in the R package `pense` with the parameter `alpha` set to 0. If a cross validated robust ridge regression is required, the corresponding function is `regmest_cv()`.

E.3.2 Robust Lasso and Elastic Net

Cohen Freue et al. (2019) describe a flexible family of robust estimators that can implement models ranging from ridge regression through elastic net to lasso by changing a single parameter α in the loss functions. They consider two loss functions:

$$\mathscr{L}_{PS}\left(\mathbf{X},\ \mathbf{y},\ \mu,\ \boldsymbol{\beta}\right)=\sigma^2\left(\mu,\ \boldsymbol{\beta}\right)+\lambda_S\left(\alpha\left\|\boldsymbol{\beta}\right\|_1+\frac{(1-\alpha)}{2}\left\|\boldsymbol{\beta}\right\|_2\right),\ \ \lambda_S>0,\ 0\leqslant\alpha\leqslant 1 \tag{E.40a}$$

$$\mathscr{L}_{PM}\left(\mathbf{X},\ \mathbf{y},\ \mu,\ \boldsymbol{\beta}\right)=\frac{1}{N}\sum_{i=1}^{N}\rho\left(\frac{\epsilon_i\left(\mu,\ \boldsymbol{\beta}\right)}{\hat{\sigma}_0}\right)+\lambda_M\left(\alpha\left\|\boldsymbol{\beta}\right\|_1+\frac{(1-\alpha)}{2}\left\|\boldsymbol{\beta}\right\|_2\right),\ \ \lambda_M>0,\ 0\leqslant\alpha\leqslant 1. \tag{E.40b}$$

The presence of the variance in (E.40a) identifies it as the loss function of an S-estimator (S stands for scale) and the presence of the ρ function in (E.40b) identifies it as an M-estimator. The corresponding estimators are known as PENSE (Penalized Elastic Net S Estimator) and PENSEM (Penalized Elastic Net S Estimator – M step). The factor of 2 in the regularization penalty and $\frac{1}{N}$ in (E.40b) are not essential - we follow the convention adopted by Cohen Freue et al. (2019) to allow easy cross-referencing. These scaling factors do not affect the solutions to (E.40a) and (E.40b), but make them easier to manipulate algebraically and interpret. The factor of 2 cancels when $\|\boldsymbol{\beta}\|_2$ is differentiated, and the factor of $\frac{1}{N}$ permits us to interpret the first term of (E.40b) as an expectation.

Kepplinger (2023) and Kepplinger and Cohen Freue (2023) describe adaptive versions of PENSE and PENSEM that reduce the bias of large coefficients and improves their ability to select variables. An initial estimate of $\boldsymbol{\beta}$ is obtained using robust ridge regression, and these coefficients are then used to weight both the magnitude and the square of the individual components of $\boldsymbol{\beta}$. The revised loss functions are

$$\mathscr{L}_{APS}\left(\mathbf{X},\, \mathbf{y},\, \mu,\, \boldsymbol{\beta}\right) = \sigma^2\left(\mu,\, \boldsymbol{\beta}\right) + \lambda_S \sum_{i=1}^{p} w_i \left(\alpha \left|\boldsymbol{\beta}_i\right| + \frac{1-\alpha}{2}\boldsymbol{\beta}_i^2\right),\ \ \lambda_S > 0,\ 0 \leqslant \alpha \leqslant 1 \tag{E.41a}$$

$$\mathscr{L}_{APM}\left(\mathbf{X},\, \mathbf{y},\, \mu,\, \boldsymbol{\beta}\right) = \frac{1}{N}\sum_{i=1}^{N} \rho\left(\frac{\epsilon_i\left(\mu,\, \boldsymbol{\beta}\right)}{\hat{\sigma}_0}\right) + \lambda_M \sum_{i=1}^{p} w_i \left(\alpha \left|\boldsymbol{\beta}_i\right| + \frac{1-\alpha}{2}\boldsymbol{\beta}_i^2\right),\ \ \lambda_M > 0,\ 0 \leqslant \alpha \leqslant 1. \tag{E.41b}$$

where

$$w_i = \left(\frac{1}{\left|\boldsymbol{\beta}_{robust\ ridge,\, i}\right|}\right)^{\gamma},\ \gamma > 0. \tag{E.42}$$

The function `adamest_cv()` in `pense` implements adaptive scaling for both lasso and elastic net. As with all regularized models, the loss function must be minimized using cross-validation, though, as the data might contain outliers, MSE is no longer a good measure of the prediction error, which must be measured using a robust estimator of scale. The cross-validation functions in `pense` allow the user to specify a range of classical and robust measures of prediction error.

E.4 Classification and Robust Logistic regression

Sometimes, we are tasked with mapping a set of features to a set of K distinct classes, or categories, of responses $C_1, C_2, \cdots, C_K$, such as the digits 0–9 ($K = 10$), or the recessionary and expansionary states of the economy that we studied in Chapter 11 ($K = 2$). The linear models we have discussed so far are ill suited to this task – we need to map their output (which we assume to be a scalar $\hat{y}$) to exactly one these K categories, to which we assign the numeric values $1, 2, \cdots, K$. This process, which is known as *classification*, can be done in a number of ways, including:

1. If the categories are naturally ordered, we can use a regularized regression algorithm to regress their numerical values against the features, and map $\hat{y}$ to a category by comparing it to a set of $K-1$ thresholds $-\infty < T_1 < T_2 < \cdots < T_{K-1} < \infty$, and output

   a. C_1 if $\hat{y} \leqslant T_1$

b. C_2 if $T_1 < \hat{y} \leqslant T_2$

c. $\vdots$

d. C_K if $T_{K-1} < \hat{y}$

2. If the categories are not naturally ordered, we can create a set of K linear models, one for each class C_j, $1 \leqslant j \leqslant K$, by associating the feature matrix $\mathbf{X}$ with a matching set of K binary response vectors $\mathbf{c}^j$, $1 \leqslant j \leqslant K$, whose i^{th} row is 1 (i.e. $\mathbf{c}_i^j = 1$) if the ith item belongs to C_j and 0 otherwise. Given a new feature vector $\mathbf{x}$, we input its values into all K models and assign the observation to C_j if $\hat{y}_j$, the fitted output of the jth model is the largest of the K fitted outputs. In essence, each one of the models makes a prediction about the class of the observation, which is then assigned to the class it most closely resembles. While attractive in theory, this method often fails in practice when $K > 2$, as some classes get masked by other classes. The masking problem can be solved using *linear discriminant analysis*, but the distributional assumptions required by it are restrictive, and we do not pursue it further.

3. If the identification problem maps naturally to one of choosing the class with the highest posterior probability, and if the sum of all the class probabilities is 1, we can apply a non-linear transformation $g(\cdot)$ to map $\hat{y}_j$ to the probability that the given features correspond to an item from C_j, for $1 \leqslant j \leqslant K$. This is most easily done using *logistic regression* when there are only two classes, which, for convenience, we label C_1 and C_2, and to which we assign values of 0 and 1 respectively. This requires only a single linear model that relates $\mathbf{X}$ to $\mathbf{c}$, the class response vector,[7] and we simplify notation by using $\hat{y}$ and $g(\hat{y})$ to denote its output and its transformed output respectively.

In the first case, both the model and the thresholds must be determined using cross-validation. Candidate models include the classical and robust regularized models described earlier in this chapter. In the second, multiple distributional assumptions, along with assumptions about the independence of measurements and the covariance of observations within classes must be made in order to operationalize it. As these assumptions are not true of most finance data sets, we do not pursue it further.

The third approach is widely used, and we focus on the $K = 2$ case to build our intuition before outlining the mathematics of the general case.

E.4.1 Logistic Regression

Logistic regression usefully bridges two observations:

1. The response of a linear model, $\hat{y}$, is unbounded on either side of the real line, and
2. Probability distributions are increasing and non-negative, and are bounded between 0 and 1.

These observations can be reconciled by mapping $\hat{y}$ to the conditional probability that an observation $\mathbf{x} = (x_1, x_2, \cdots, x_p)'$ is associated with C_2 via an appropriate *link function* $g(\cdot)$. Cumulative distribution functions are well suited to use as link functions as they naturally map the real line to $[0, 1]$, and we therefore write[8]

[7] $\mathbf{c}_i = 0$ if the ith row of $\mathbf{X}$ is associated with C_1 and is 1 if it is associated with C_2.

[8] While E.43 maps $\hat{y}$ to $p(\mathbf{x})$ via $g(\cdot)$, it is equally common to see $p(\mathbf{x})$ being mapped to $\hat{y}$ via $g^{-1}(\cdot)$.

$$\begin{aligned} g(\hat{y}) &= F(\hat{y}, \boldsymbol{\theta}), \; -\infty < \hat{y} < \infty, \\ &= Pr[\mathbf{x} \in C_2] \\ &\triangleq p(\mathbf{x}). \end{aligned} \tag{E.43}$$

$F(\cdot)$ takes as its inputs a vector of parameters $\boldsymbol{\theta}$, as well as $\hat{y} = \mathbf{x}'\boldsymbol{\beta}$, where $\mathbf{x}$ is a column vector of features associated with the observation and $\boldsymbol{\beta}$ is the coefficient vector generated by the linear model. Finally, $p(\mathbf{x})$ is the probability that this observation is associated with C_2. [9] The model must be estimated by optimizing over $F(\cdot)$, $\boldsymbol{\theta}$ and $\boldsymbol{\beta}$ using maximum likelihood estimation and/or cross-validation.

Allowing complete freedom in the choice of $F(\cdot)$ so underconstrains the problem that no unique solution can be found. In practice, therefore, we restrict the allowable family of distributions. The two most widely used families are the Normal (in which case the model is known as a *probit* model) or the *logit* (in which case it is known as logistic regression), and it is the second that has come to dominate usage, largely on account of its lower computational complexity. With appropriate scaling, the normal and logit densities almost overlap each other, so the results derived from their use are almost identical.

Logistic regression starts by mapping $p(\mathbf{x})$ to the entire real line using the logit, or the quantile function of the standard logistic distribution:

$$\begin{aligned} \hat{y} &= \mathbf{x}'\boldsymbol{\beta} \\ &= \ln\left(\frac{p(\mathbf{x})}{1 - p(\mathbf{x})}\right). \end{aligned} \tag{E.44}$$

The bracketed term on the right hand side of (E.44) is known as the *odds* (the entire right hand side is known as the *log odds*) and can take any positive value. It is easily seen that

$$p(\mathbf{x}) = \frac{\text{odds}}{1 + \text{odds}}, \tag{E.45}$$

and (E.44) is easily inverted to give

$$\begin{aligned} p(\mathbf{x}) &= g(\hat{y}) \\ &= \frac{\exp(\hat{y})}{1 + \exp(\hat{y})} \\ &= \frac{\exp(\mathbf{x}'\boldsymbol{\beta})}{1 + \exp(\mathbf{x}'\boldsymbol{\beta})} \\ &= \frac{1}{1 + \exp(-\mathbf{x}'\boldsymbol{\beta})}. \end{aligned} \tag{E.46}$$

When we are given a feature matrix $\mathbf{X}$ whose i^{th} row is $\mathbf{X}_i$, we correspondingly write

[9] Correspondingly, $1 - p(\mathbf{x})$ is the probability that it is associated with C_1.

$$
\begin{aligned}
p(\mathbf{X}_i) &= g(\hat{y}_i) \\
&= \frac{\exp(\hat{y}_i)}{1+\exp(\hat{y}_i)} \\
&= \frac{\exp(\mathbf{X}_i\,\boldsymbol{\beta})}{1+\exp(\mathbf{X}_i\,\boldsymbol{\beta})} \\
&= \frac{1}{1+\exp(-\mathbf{X}_i\,\boldsymbol{\beta})}.
\end{aligned} \tag{E.47}
$$

If the rows of $\mathbf{X}$ are independent, we can use the fact that $\mathbf{c}_i$ is binary to write the log-likelihood function of $\boldsymbol{\beta}$ as

$$
\begin{aligned}
l(\boldsymbol{\beta}) &= \ln\, L(\boldsymbol{\beta}) \\
&= \ln\left(\prod_{i=1}^{N}\left(p(\mathbf{X}_i)\right)^{\mathbf{c}_i}\left(1-p(\mathbf{X}_i)\right)^{1-\mathbf{c}_i}\right) \\
&= \sum_{i=1}^{N}\mathbf{c}_i\cdot\ln p(\mathbf{X}_i)\;+\;(1-\mathbf{c}_i)\cdot\ln\left(1-p(\mathbf{X}_i)\right) \\
&= \sum_{i=1}^{N}\mathbf{c}_i\cdot\ln\left(\frac{p(\mathbf{X}_i)}{1-p(\mathbf{X}_i)}\right)\;+\;\sum_{i=1}^{N}\cdot\ln\left(1-p(\mathbf{X}_i)\right) \\
&= \sum_{i=1}^{N}\mathbf{c}_i\cdot\mathbf{X}_i\,\boldsymbol{\beta}\;-\;\sum_{i=1}^{N}\ln\left(1+\exp(\mathbf{X}_i\,\boldsymbol{\beta})\right).
\end{aligned} \tag{E.48}
$$

It is easily shown that

$$
\begin{aligned}
\nabla l(\boldsymbol{\beta}) &= \frac{\partial l(\boldsymbol{\beta})}{\partial\boldsymbol{\beta}} \\
&= \sum_{i=1}^{N}\left(\mathbf{c}_i-\frac{\exp(\mathbf{X}_i\,\boldsymbol{\beta})}{1+\exp(\mathbf{X}_i\,\boldsymbol{\beta})}\right)\mathbf{X}_i'
\end{aligned} \tag{E.49}
$$

and

$$
\begin{aligned}
\mathbf{H}(\boldsymbol{\beta}) &= \frac{\partial^2 l(\boldsymbol{\beta})}{\partial\boldsymbol{\beta}\partial\boldsymbol{\beta}'} \\
&= -\sum_{i=1}^{N}\mathbf{X}_i'\mathbf{X}_i\,\frac{\exp(\mathbf{X}_i\,\boldsymbol{\beta})}{\left(1+\exp(\mathbf{X}_i\,\boldsymbol{\beta})\right)^2}.
\end{aligned} \tag{E.50}
$$

We identify $\boldsymbol{\beta}_{Logistic\ Regression}$ by setting

$$
\nabla l(\boldsymbol{\beta}) = \mathbf{0}, \tag{E.51}
$$

and then using gradient descent to iteratively identify the value of $\boldsymbol{\beta}$ that solves (E.51). The relevant details can be found in Section 4.4.1 of Hastie, R. Tibshirani, and J. H. Friedman (2009).

Logistic regression, like the other linear models we earlier described, can be regularized, and `glmnet` allows the addition of an elastic net penalty to (E.48). The corresponding loss function is

$$\mathscr{L}_{LR}(\mathbf{X},\ \mathbf{y},\ \mu,\ \boldsymbol{\beta}) = -\frac{1}{N}\, l(\boldsymbol{\beta}) + \lambda_{LR}\left(\alpha\, \|\boldsymbol{\beta}\|_1 + \frac{(1-\alpha)}{2}\, \|\boldsymbol{\beta}\|_2\right),\ \ \lambda_{LR} \geqslant 0,\ 0 \leqslant \alpha \leqslant 1$$

Negating (E.48) and dividing it by N turns the log-likelihood into an expected loss, and the addition of the elastic net penalty allows for variable selection and shrinkage. When $\lambda_{LR} = 0$, the loss function corresponds to classical logistic regression. As λ_{LR} increases, the choice of alpha determines whether the regularization is ridge-like, lasso-like or elastic net-like.

It proves useful to routinely include a small ridge penalty when running logistic regression to protect against divergence in the special case when the classes are completely separated and have no overlap. In this case, the MLE of $\boldsymbol{\beta}$ is not unique, and some of its entries will become infinite. A small ridge penalty protects against this.

Robust logistic regression is subtly different from robust linear regression, as the respones fall into a finite number of buckets, so that while errors may exist in some responses, no outliers can exist. The features, on the other hand, are unbounded, and can act as *leverage points* if they lie far away from the majority of the features.

The influence function of (E.51), the MLE for Logistic Regression, can be shown to be (see Section 7.1 of Maronna et al. (2019) or Jann (2020))

$$IF(\mathbf{c},\ \mathbf{X},\ \boldsymbol{\beta}) = \left(\frac{1}{N}\sum_{i=1}^{N} \mathbf{X}_i'\mathbf{X}_i\, \frac{\exp(\mathbf{X}_i\,\boldsymbol{\beta})}{(1+\exp(\mathbf{X}_i\,\boldsymbol{\beta}))^2}\right)^{-1} \cdot \sum_{i=1}^{N}\left(\mathbf{c}_i - \frac{\exp(\mathbf{X}_i\,\boldsymbol{\beta})}{1+\exp(\mathbf{X}_i\,\boldsymbol{\beta})}\right)\mathbf{X}_i'. \tag{E.52}$$

As the first term $\longrightarrow \left(E\left(\mathbf{x}'\mathbf{x}\,\frac{\exp(\mathbf{x}\boldsymbol{\beta})}{(1+\exp(\mathbf{x}\boldsymbol{\beta}))^2}\right)\right)^{-1}$ where $\mathbf{x}$ represents a random row of $\mathbf{X}$, and as the magnitude of bracketed term in the sum is bounded by 1, the primary determinant of the impact of an added observation on $\boldsymbol{\beta}$ is $\mathbf{X}_i$. Robust logistic regression applies a data-dependent weight $\mathbf{W}_i$ that depends on how far $\mathbf{X}_i$ is from its distribution to the estimating equation (E.51), which now becomes

$$\sum_{i=1}^{N} \mathbf{W}_i\left(\mathbf{c}_i - \frac{\exp(\mathbf{X}_i\,\boldsymbol{\beta})}{1+\exp(\mathbf{X}_i\,\boldsymbol{\beta})}\right)\mathbf{X}_i' = \mathbf{0}, \tag{E.53}$$

where the vector of weights $\mathbf{W}$ is derived from a robust estimate of the Mahalonobis distances of the rows of $\mathbf{X}$ from their common distribution. Points that lie further away from the distribution are down-weighted so as not to distort the estimate of $\boldsymbol{\beta}$. The details are complex and can be found in Section 7.2 of Maronna et al. (2019). The R package `RobstatTM` implements the weighted robust logistic regression algorithm `logregWBY` that was first described in Bianco and Yohai (1996) and further enhanced in Croux and Haesbroeck (2003).

E.5 Projection Pursuit Regression and Neural Networks

E.5.1 Projection Pursuit

Least squares and its derivatives provide powerful ways by which to identify structure in a high-dimensional space by optimally projecting features onto a low-dimensional subspace, most often a

hyperplane. But what if the features can be made more visually informative by first scaling and rotating, and perhaps even discarding, some of their components before projecting them? Or if they exhibit clustering when projected onto some special hyperplane?

Figure E.2 illustrates the nature of the problem. The data plotted is a mixture of two independent random variables Z_1 and Z_2. Z_1 is normally distributed with mean 1.5 and variance 1, while Z_2 is exponential with mean 1. The plotted variables X and Y are derived from Z_1 and Z_2 as follows:

$$\begin{aligned} X &= Z_1 \text{ w.p. } \frac{2}{3},\ 1 - Z_2 \text{ w.p. } \frac{1}{3} \\ Y &= Z_1 \text{ w.p. } \frac{2}{3}, \qquad\quad - Z_2 \text{ w.p. } \frac{1}{3} \end{aligned}$$

The scatter plot at the top left displays the actual data, while the density plot at the top right clearly shows the concentration of the points. The separation between the two distributions is clear, and a 45° line nicely separates the two distributions. The density of X (bottom left), however, shows only a single peak, and that of Y (bottom right), identifies both peaks, but suggests that the two distributions are almost equally likely.

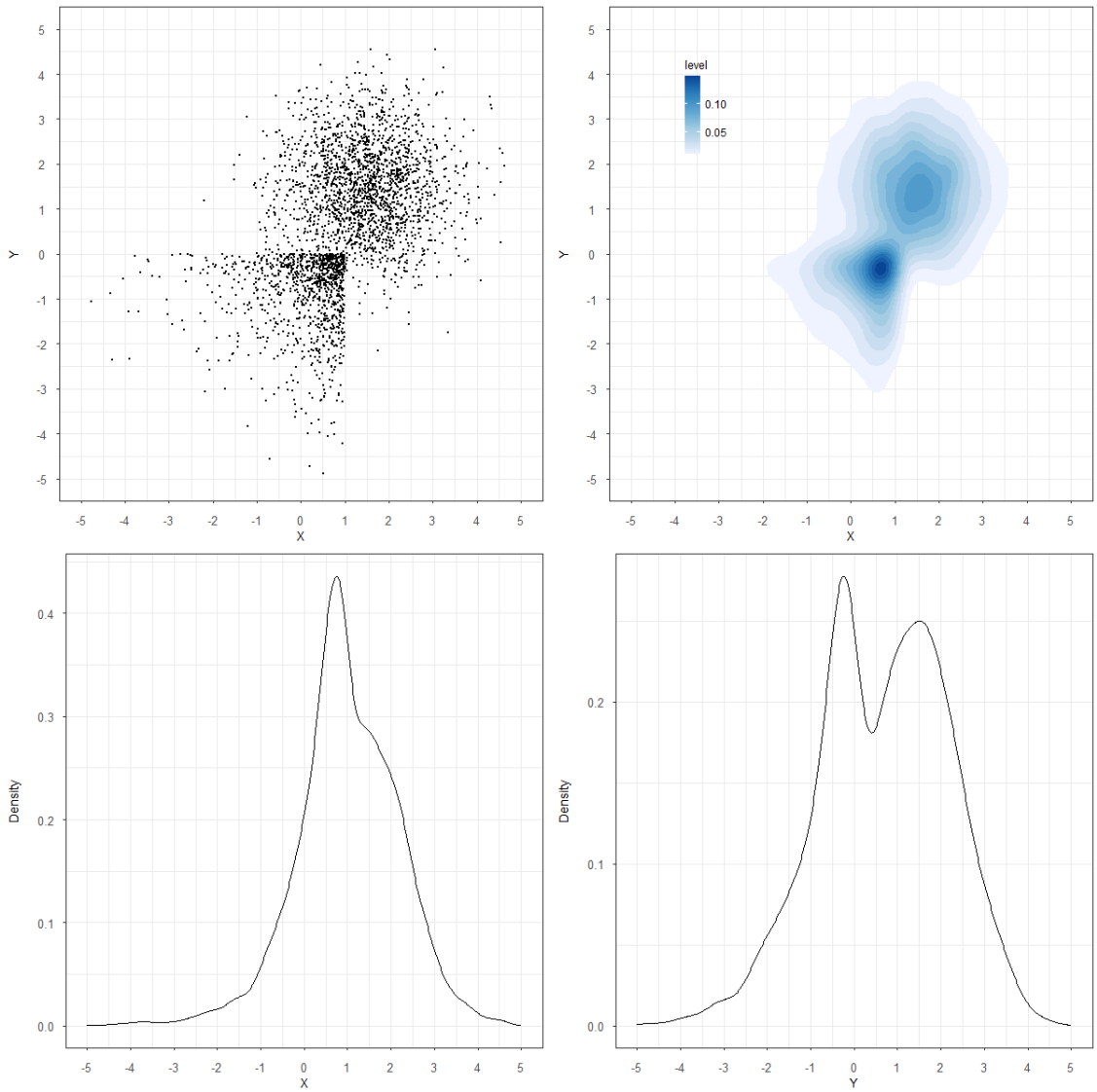

Fig. E.2 Four views of two variables and their densities

Projection pursuit[10] is the process of exposing structure in a feature matrix beyond its covariance matrix by maximizing an appropriately chosen *projection index* I. The value of the projection index ought to reflect how interesting a particular projection of the features might be to an analyst. The notion of interestingness is application dependent, but is often interpreted as how non-normal the distribution of the projected data is.

One-dimensional projection pursuit searches for a vector $\boldsymbol{\beta}_1$ that best exposes the underlying structure of $\mathbf{X}$ by maximizing $I\left(\mathbf{X}'\boldsymbol{\beta}\right)$. Two-dimensional projection pursuit adds a second projection vector $\boldsymbol{\beta}_2$ as well as a bivariate projection index $I\left(\mathbf{X}'\boldsymbol{\beta}_1, \mathbf{X}'\boldsymbol{\beta}_2\right)$ that is maximized when interesting structure in the joint density of $\mathbf{X}$ in two orthogonal directions is best exposed.

Projection pursuit is often used to maximally separate clusters in a feature matrix $\mathbf{X}$, and so lends itself to both supervised and unsupervised learning. Projection pursuit regression goes a step further – it

[10] The catchy name appears to have been first mentioned in J. H. Friedman and Tukey (1974).

applies nonlinear transformations to projections of data on a collection of identified proejction vectors to obtain a better regression surface.

The R packages `Pursuit`, `tourr` and `REPPlab` allow one to explore and visualize various projections of a data set, and Figure E.3 displays two projections of the famous `iris` dataset along two axes using `Pursuit::GrandTour`.[11] The plot on the left displays the raw data projected without any attempt to maximize separation between classes, and evidences significant overlap between the three species. The plot on the right, in contrast, shows how an optimal projection allows error-free identification of all irises of species setosa, and excellent, though not perfect, separation of the two other species (versicolor and virginica.)

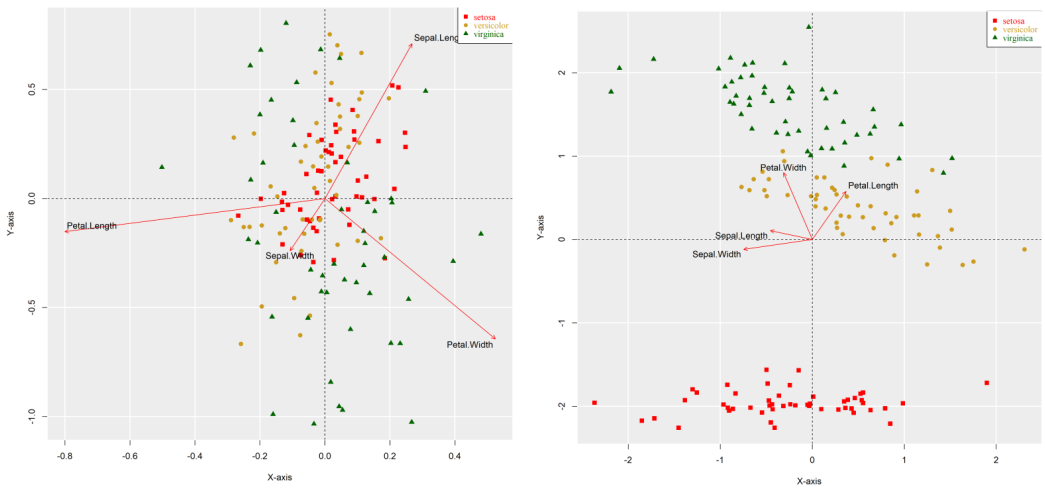

Fig. E.3 Initial and final projections of the Iris dataset

The projection vectors employed for the right-hand plot are displayed in Table E.1, and it is easily confirmed that they are orthogonal and have unit length. Projection pursuit can be performed recursively – all irises of class setosa can be removed from the dataset, and the algorithm can then be applied to separate irises of the two the remaining classes (versicolor and virginica) as well as possible.

Table E.1 Unit projection vectors for Figure E.3

	Projection Vector 1 (X-axis)	Projection Vector 2 (Y-axis)
Sepal.Length	-0.4567710	0.1068818
Sepal.Width	-0.7476539	-0.1170979
Petal.Length	0.3671295	0.5765450
Petal.Width	-0.3123937	0.8015362

Kruskal (1969) initiated work on projection pursuit by searching for clusters that maximize an "index of condensation" which he defined to be the coefficient of variation of the inter-point distances generated by $\mathbf{X\beta}$, i.e.

[11] See `https://en.wikipedia.org/wiki/Iris_flower_data_set` for the history of this dataset, which is widely used in machine learning. It contains 4 measurements (the length and the width of petals and sepals) on 150 flowers representing three species of irises. The measurements were made by Edgar Anderson and the data set was made famous by Sir Ronald Fisher in Fisher (1936).

$$I_{Kruskal} = \frac{\sigma_{y_i - y_j}}{\overline{y_i - y_j}}. \tag{E.54}$$

The intuition that motivates (E.54) is simple: if projecting $\mathbf{X}$ onto $\boldsymbol{\beta}$ results in a few well-separated clusters of points, the average inter-point distance within clusters will be small, while the average inter-point distance across clusters will be large, simultaneously increasing its numerator and decreasing its denominator. Unfortunately $I_{Kruskal}$ is not robust to outliers, though it can be improved using robust estimates of both the numerator and denominator.

Over the years, many such indices have been identified, and Hou and Wentzell (2011) use kurtosis as an index to identify the directions $\mathbf{u}_i$ that maximize cluster separation by *minimizing* the kurtosis of $\mathbf{u}_i \cdot \mathbf{y}$. Their technique rests on Moors (1986) observation that kurtosis measures the dispersion of x around both $\mu - \sigma$ and $\mu + \sigma$, making its minimization a simple way by which to optimally separate observations into two classes, one centered at $\mu - \sigma$ and the other at $\mu + \sigma$.

$$\begin{aligned}
\kappa &= E\left[\left(\frac{x - \mu}{\sigma}\right)^4\right] \\
&= E\left[\left(\left(\frac{x - \mu}{\sigma}\right)^2\right)^2\right] \\
&= E\left[\left(\left(\frac{x - \mu}{\sigma}\right)^2\right)\right]^2 + \operatorname{var}\left(\left(\frac{x - \mu}{\sigma}\right)^2\right) \\
&= 1 + \operatorname{var}\left(\left(\frac{x - \mu}{\sigma}\right)^2\right)
\end{aligned} \tag{E.55}$$

Figure E.4 and Table E.2 illustrates how the Iris dataset can be projected onto a single vector and separated by minimizing kurtosis using the function `Pursuit::PP_Optimizer`. The results are virtually identical to those in Figure E.3.

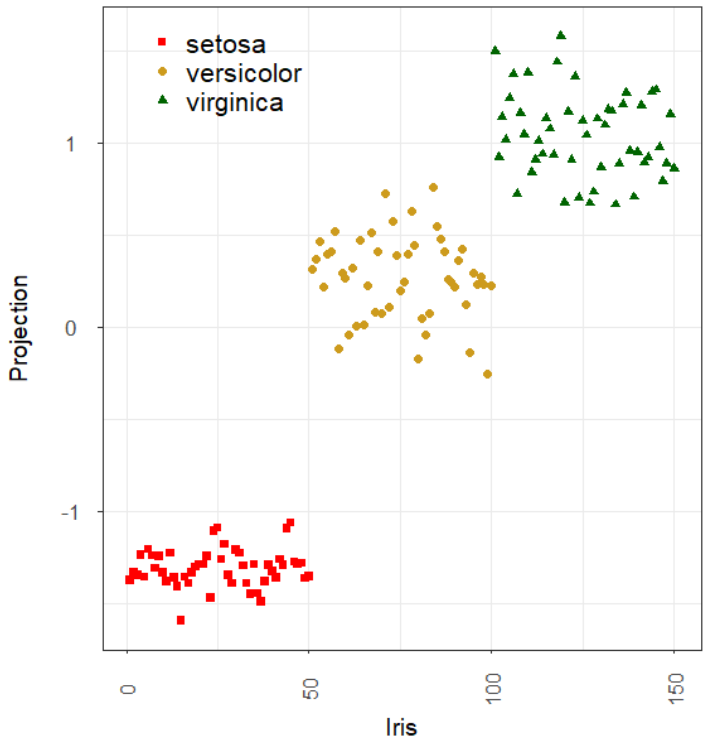

Fig. E.4 Projections onto the kurtosis minimizing projection vector

Table E.2 The kurtosis minimizing projection vector

Feature	Projection Vector
Sepal.Length	0.988068380
Sepal.Width	0.114840009
Petal.Length	-0.102380058
Petal.Width	0.007139457

Projection pursuit analysis (PPA) has much in common with Independent Components Analysis (ICA), which in turn is related to Principal Components Analysis (PCA). The links between the techniques are addressed briefly in Chapter 14 of Hastie, R. Tibshirani, and J. H. Friedman (2009) and are developed comprehensively in Hyvärinen and Oja (2000) and Hyvärinen, Karhunen, et al. (2001).

E.5.2 Projection Pursuit Regression

J. l. Friedman and Stuetzle (1981) use the projections identified by PPA to usefully generalize Least Squares[12] by writing

[12] See Appendix B for a development of Least Squares.

$$\hat{y} = \overline{y} + \sum_{i=1}^{m} f_i\left(\mathbf{x}'\boldsymbol{\beta}_i\right), \tag{E.56}$$

where $\boldsymbol{\beta}_i$ is the ith projection,[13] and $f_i,\ 1 \leqslant i \leqslant m$ are smooth *ridge functions*, or functions that vary only in the direction of $\boldsymbol{\beta}$.[14] The ridge functions in (E.56)are not restricted to be linear, allowing it to capture nonlinearities in the relationship between $\mathbf{x}$ and y. Most importantly, Diaconis and Shahshahani (1984) show that ridge functions possess a universal approximation property – any function of p variables can be appproximated arbitrarily well by ridge functions for sufficiently large m, echoing the Stone-Weierstrass Theorem for continuous functions on Hausdorff spaces.

J. l. Friedman and Stuetzle (1981) call their method Projection Pursuit Regression (PPR) and present an iterative algorithm for identifying m, f_i and $\boldsymbol{\beta}_i$. Even though the algorithm is refined in J. H. Friedman (1985) and J. H. Friedman (1987), we present only the original version, both for ease of exposition and for relevance. In spite of its distinguished parentage, PPR is no longer widely used, and lives on primarily as the intellectual predecessor of neural networks, largely on account of the universal approximation property they share.

Algorithm E.1 The Friedman-Steutzle algorithm for Projection Pursuit Regression

1. `Initialize:`
   a. $i \longleftarrow 1$
   b. $m \longleftarrow m_{max}$
   c. `Demean the response vector` $\mathbf{y}_1 \longleftarrow \mathbf{y} - \bar{\mathbf{y}}\mathbf{1}$
2. `While` $i = 1,\ 2, \cdots,\ m$ `and` (res^2> `Threshold`):
   a. `Compute the residuals` $\epsilon_i \longleftarrow \mathbf{y}_i - \sum_{j=1}^{i-1} f_j\left(\mathbf{X}\boldsymbol{\beta}_j\right)$
   b. `Search for and dentify the` i^{th} `projection vector` $\boldsymbol{\beta}_i$ `and ridge function` f_i `that minimizes the sum of squared residuals` $res^2 = \left(\epsilon_i - f_i\left(\mathbf{X}\boldsymbol{\beta}_i\right)\right)'\left(\epsilon_i - f_i\left(\mathbf{X}\boldsymbol{\beta}_i\right)\right)$, where $f_i(\cdot)$is applied to each component of $\mathbf{X}\boldsymbol{\beta}_i$.

Algorithm E.1 is a forward stagewise procedure – once a projection vector is identified, it is not modified in later stages. In contrast, the SMART algorithm decribed in J. H. Friedman (1985) directly minimizes the sum of squared residuals by iteratively minimizing the magnitude of the residual (E.56) using a Gauss-Newton iteration

$$m^*,\ \mathbf{f}^*,\ \boldsymbol{\beta}^* = \underset{m,\ f_i,\ \boldsymbol{\beta}_i}{argmin}\left(\mathbf{y} - \bar{\mathbf{y}}\mathbf{1} - \sum_{i=1}^{m} f_i\left(\mathbf{X}\boldsymbol{\beta}_i\right)\right)'\left(\mathbf{y} - \bar{\mathbf{y}}\mathbf{1} - \sum_{i=1}^{m} f_i\left(\mathbf{X}\boldsymbol{\beta}_i\right)\right), \tag{E.57}$$

where m^*, $\mathbf{f}^*$ and $\boldsymbol{\beta}^*$ are the optimal values of m, $f_i,\ 1 \leqslant i \leqslant m$ and $\boldsymbol{\beta}_i,\ 1 \leqslant i \leqslant m$.

The two solutions can be strikingly different, particularly when the features are almost collinear. The SMART algorithm is available as the Base R functions `ppr` and `plot.ppr`.

Table E.3 displays the projection vectors identified by PPR as well as the LS fits of Fama-French 3– and 4–factor models to the weekly returns of the stock with ticker FNB in 2008. It echoes the comparison between LS and robust regression in Table 6.9. We have deliberately suppressed the intercept as it is small, and as the projection vectors do not have an intercept. The first four columns display the first

[13] These may not be identical to the projections identified by projection pursuit.

[14] See Konyagin et al. (2018) for the theory of ridge functions.

two projection vectors for PPR for the 3– and 4–factor Fama-French models while the last two columns display LS fits using the same data. It is not possible to display or compactly characterize the entire PPR surface as it involves non-linear smoothing of the residuals generated by each of the ridge functions.

Table E.3 Projection Pursuit Vectors and LS fits of FF3 and FFC4 models to FNB 2008 weekly returns.

```
## Error in column_spec(column_spec(column_spec(kable_styling(kable(projVectorsAndCoeffs, : could not find
function "column_spec"
```

Table E.4 displays the residual sums of squares for PPR and LS fits to the weekly returns of FNB in 2008. PPR roughly halves the error variance, in part due to the fact that it fits the data with non-linear ridge functions. It is striking that both fits see little improvement when moving from three factors to four, supporting the widely-held view that equity returns are explained in large part by a few systematic factors.

Table E.4 Residual Sum of Squares for PPR and LS fits of FF3 and FFC4 models to FNB 2008 weekly returns.

```
## Error in column_spec(kable_styling(kable(df, format = "latex", booktabs = TRUE, : could not find function
"column_spec"
```

References

Akaike, Hirotugu (1974). “A new look at the statistical model identification”. In: *IEEE Transactions on Automatic Control* 19.6, pp. 716–723. ISSN: 0018-9286. DOI: 10.1109/tac.1974.1100705.

Bertsimas, Dimitris, King, Angela, and Mazumder, Rahul (2016). “Best Subset Selection via a Modern Optimization Lens”. In: *The Annals of Statistics* 44.2, pp. 813–852.

Bianco, Ana M. and Yohai, Victor J. (1996). “Robust Estimation in the Logistic Regression Model”. In: *Robust Statistics, Data Analysis, and Computer Intensive Methods*. Ed. by Helmut Rieder. Lecture Notes in Statistics. Springer, pp. 17–34.

Breiman, Leo (1996). “Heuristics of Instability and Stabilization in Model Selection”. In: *The Annals of Statistics* 24.6, pp. 2350–2383.

– (2001). “Statistical Modeling: The Two Cultures”. In: *Statistical Science* 16.3, pp. 199–215.

Breiman, Leo and Spector, Philip (1992). “Submodel Selection and Evaluation in Regression. The X-Random Case”. In: *International Statistical Review* 60.3, pp. 291–319.

Cao, Sean et al. (2024). “From Man vs. Machine to Man + Machine: The Art and AI of Stock Analyses”. In: *Journal of Financial Economics* 158.

Cohen Freue, Gabriela V. et al. (2019). “Robust Elastic Net Estimators for Variable Selection and Identification of Proteomic Biomarkers”. In: *The Annals of Applied Statistics* 13.4, pp. 2065–2090. DOI: 10.1214/19-AOAS1269.

Croux, Cristophe and Haesbroeck, Gentiane (2003). “Implementing the Bianco and Yohai estimator for logistic regression”. In: *Computational Statistics and Data Analysis* 43 (1-2), pp. 273–295.

deLeeuw, Jan (1992). *Introduction to Akaike (1973) Information Theory and an Extension of the Maximum Likelihood Principle*. DOI: 10.1007/978-1-4612-0919-5_37.

Diaconis, Persi and Shahshahani, Mehrdad (1984). “On Nonlinear Functions of Linear Combinations”. In: *SIAM Journal on Scientific and Statistical Computing* 5 (1), pp. 175–191.

Efron, Bradley et al. (2004). “Least Angle Regression”. In: *The Annals of Statistics* 32.2, pp. 407–499.

Fan, Jianqing and Li, Runze (2001). “Variable selection via nonconcave penalized likelihood and its oracle properties”. In: *Journal of the American Statistical Association* 96.456, pp. 1348–1360.

Fan, Jianqing, Ma, Cong, and Zhong, Yiqiao (2021). “A selective overview of deep learning”. In: *Statistical Science* 36.2, pp. 264–290.

Fisher, Ronald A. (1936). “The use of multiple measurements in taxonomic problems”. In: *Annals of Eugenics* 7.7, pp. 179–188.

Friedman, Jerome H. (1985). *Classification and Multiple Regression Through Projection Pursuit*. Tech. rep. Stanford University.

– (1987). “Exploratory Projection Pursuit”. In: *Journal of the American Statistical Association* 82.397, pp. 249–266.

Friedman, Jerome H. and Tukey, John W. (1974). “A Projection Pursuit Algorithm for Exploratory Data Analysis”. In: *IEEE Transactions on Computers, Series C* 23.9, pp. 881–889.

Friedman, Jerome lH. and Stuetzle, Werner (1981). “Projection Pursuit Regression”. In: *Journal of the American Statistical Association* 76.376, pp. 817–823.

Gu, Shihao, Kelly, Bryan, and Xiu, Dacheng (2020). "Empirical Asset Pricing via Machine Learning". In: *The Review of Financial Studies* 33.5, pp. 2223–2273.

Hastie, Trevor (2020). "Ridge Regularization: An Essential Concept in Data Science". In: *Technometrics* 62.4, pp. 426–433.

Hastie, Trevor, Tibshirani, Robert, and Friedman, Jerome H. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. Springer: New York, NY.

Hastie, Trevor, Tibshirani, Robert, and Tibshirani, Ryan J. (2020). "Best Subset, Forward Stepwise or Lasso? Analysis and Recommendations Based on Extensive Comparisons". In: *Statistical Science* 35.4, pp. 579–592.

Hoerl, Arthur E. and Kennard, Robert W. (1970). "Ridge Regression: Biased Estimation for Nonorthogonal Problems". In: *Technometrics* 12.1, pp. 55–67.

Hou, Siyuan and Wentzell, Peter (2011). "Fast and simple methods for the optimization of kurtosis used as a projection pursuit index". In: *Analytica Chimica Acta* 704, pp. 1–15.

Hyvärinen, Aapo, Karhunen, Juha, and Oja, Erkki (2001). *Independent Component Analysis*. Wiley.

Hyvärinen, Aapo and Oja, Erkki (2000). "Independent Component Analysis: Algorithms and Applications". In: *Neural Networks* 13 (4-5), pp. 411–430.

James, Gareth et al. (2013). *An Introduction to Statistical Learning*. Springer New York. DOI: `10.1007/978-1-4614-7138-7`.

Jann, Ben (2020). *Influence functions continued. A framework for estimating standard errors in reweighting, matching, and regression adjustment*. Tech. rep. University of Bern, Department of Social Sciences. URL: `https://econpapers.repec.org/paper/bsswpaper/35.htm`.

Johnstone, Iain M. and Titterington, Michael D. (2009). "Statistical challenges of high-dimensional data". In: *Philosophical Transactions of the Royal Society A* 367, pp. 4237–4253.

Kelly, Bryan and Xiu, Dacheng (2023). "Financial Machine Learning". In: *SSRN*. URL: `https://papers.ssrn.com/abstract_id=4501707`.

Kepplinger, David (2023). "Robust variable selection and estimation via adaptive elastic net S-estimators for linear regression". In: *Computational Statistics and Data Analysis* 183, pp. 1–13.

Kepplinger, David and Cohen Freue, Gabriela V. (2023). "Robust Prediction and Protein Selection with Adaptive PENSE". In: *Statistical Analysis of Proteomic Data*. Ed. by Thomas Burger. Springer. Chap. 14, pp. 315–332.

Kim, Alex, Muhn, Maximilian, and Nikolaev, Valeri V. (2023). "From Transcripts to Insights: Uncovering Corporate Risks Using Generative AI". In: *SSRN: https://ssrn.com/abstract=4593660*.

– (2024a). "Bloated Disclosures: Can ChatGPT Help Investors Process Information?" In: *SSRN*. URL: `https://ssrn.com/abstract=4425527`.

– (2024b). "Financial Statement Analysis with Large Language Models". In: *SSRN*.

Konyagin, S. V., Kuleshov, A. A., and Maiorov, V. E. (2018). "Some Problems in the Theory of Ridge Functions". In: *Proceedings of the Steklov Institute of Mathematics* 301, pp. 144–169.

Kruskal, Joseph B. (1969). "Toward a practical method which helps uncover the structure of a set of observations by finding the line transformation which optimizes a new "index of condensation"". In: *Statistical Computation*. Ed. by Roy C. Milton and John A. Nelder. Academic Press, pp. 427–440.

Maronna, Ricardo A. (2011). “Robust Ridge Regression for High–Dimensional Data”. In: *Technometrics* 53.1, pp. 44–53.

Maronna, Ricardo A. et al. (2019). *Robust Statistics: Theory and Methods (with R).* 2nd ed. John Wiley & Sons.

Martin, R. Douglas, Guerard, John B., and Xia, Daniel Z. (2024). “Earnings to Price Analysis with mOpt Versus Bisquare RobustRegression”. In: *SSRN*. URL: `:%20https://ssrn.com/abstract=4777975`.

Martin, R. Douglas, Stoyanov, Stoyan V., et al. (2023). “Robust Statistics for Portfolio Construction and Analysis”. In: *Journal of Portfolio Management* 49.9, pp. 105–139.

Moors, J. J. A. (1986). “The Meaning of Kurtosis: Darlington Reexamined”. In: *The American Statistician* 40.4, pp. 283–284.

Schwarz, Gideon (1978). “Estimating the Dimension of a Model”. In: *The Annals of Statistics* 6.2, pp. 461–464. ISSN: 0090-5364. DOI: `10.1214/aos/1176344136`.

Theobald, C. M. (1974). “Generalizations of Mean Square Error Applied to Ridge Regression”. In: *Journal of the Royal Statistical Society, Series B* 36.1, pp. 103–106.

Tibshirani, Robert (1996). “Regression shrinkage and selection via the lasso”. In: *Journal of the Royal Statistical Society B* 58, pp. 267–288.

– (2011). “Regression shrinkage and selection via the lasso: a retrospective”. In: *Journal of the Royal Statistical Society B* 73 (Part 3), pp. 273–282.

Tibshirani, Ryan J. (2013). “The lasso problem and uniqueness”. In: *Electronic Journal of Statistics* 7, pp. 1456–1490.

Wieringen, Wessel N. van (2023). *Lecture Notes on Ridge Regression*. Tech. rep. Department of Mathematics, Vrije Universiteit Amsterdam. URL: `https://arxiv.org/pdf/1509.09169.pdf`.

Yuan, Ming and Lin, Yi (2006). “Model selection and estimation in regression with grouped variables”. In: *Journal of the Royal Statistical Society B* 68 (Part 1), pp. 49–67.

Zou, Hui (2006). “The Adaptive Lasso and its Oracle Properties”. In: *Journal of the American Statistical Association* 101.476, pp. 1418–1429.

Zou, Hui and Hastie, Trevor (2005). “Regularization and variable selection via the elastic net”. In: *Journal of the Royal Statistical Society B* 67.Part 2, pp. 301–320.

Zou, Hui and Zhang, Hao Helen (2009). “On the Adaptive Elastic Net With a Diverging Number of Parameters”. In: *The Annals of Statistics* 37.4, pp. 1733–1751.